Git Command Notebook

Commands are grouped by task. Replace FILE, BRANCH, HASH, URL, and EMAIL with your own values; they are placeholders, not literal arguments. Run commands inside your repository unless creating or cloning one. This reference includes practical command forms plus the complete command catalogue reported by the installed Git version. It does not enumerate every possible option combination. Run git help COMMAND for its full documentation.

1. Help And Configuration

COMMAND	MEANING
`git --version`	*Show your Git version.*
`git help`	*Show basic help.*
`git help -a`	*List available commands and help topics.*
`git help -g`	*List concept guides.*
`git help commit`	*Open a command's manual.*
`git commit -h`	*Show short command help.*
`git config --global user.name "YOUR_NAME"`	*Set the name attached to commits.*
`git config --global user.email "EMAIL"`	*Set the email attached to commits.*
`git config --global init.defaultBranch main`	*Use main for new repositories.*
`git config user.name "YOUR_NAME"`	*Override the name for this repo only.*
`git config --list --show-origin`	*Show configuration and its sources.*
`git config --global core.editor "code --wait"`	*Use VS Code for commit messages.*
`git config --global alias.st "status"`	*Make git st an alias for git status.*

2. Create Or Copy A Repository

COMMAND	MEANING
`git init`	*Initialize a repository; does not stage files.*
`git init -b main`	*Initialize with main as the first branch.*
`git clone URL`	*Clone a repository and configure origin.*
`git clone URL folder`	*Clone into a chosen folder.*
`git clone -b BRANCH URL`	*Clone and check out a chosen branch.*
`git clone --depth 1 URL`	*Make a shallow clone with limited history.*
`git clone --recurse-submodules URL`	*Clone and initialize nested submodules.*

COMMAND	MEANING
`git clone --bare URL`	*Clone without a working directory.*

3. Check, Stage, And Commit

COMMAND	MEANING
`git status`	*Show staged, unstaged, and untracked changes.*
`git status -sb`	*Show a compact status and branch summary.*
`git add FILE`	*Stage the current content of one file.*
`git add .`	*Stage all changes under the current directory.*
`git add -A`	*Stage additions, modifications, and deletions repo-wide.*
`git add -p`	*Select changes interactively, one hunk at a time.*
`git diff`	*Show unstaged changes to tracked files.*
`git diff --staged`	*Show changes prepared for the next commit.*
`git diff HEAD`	*Compare tracked working content with HEAD.*
`git diff --check`	*Check a diff for whitespace errors.*
`git commit -m "MESSAGE"`	*Commit the staged snapshot locally.*
`git commit -am "MESSAGE"`	*Stage modified/deleted tracked files and commit; does not add new untracked files.*
`git commit --amend`	*Replace the latest commit and edit its message.*
`git commit --amend --no-edit`	*Replace it using staged changes; keep its message.*
`git mv old.txt new.txt`	*Rename a tracked file and stage the rename.*
`git rm FILE`	*Remove a tracked file and stage the deletion.*
`git rm --cached FILE`	*Stop tracking a file; keep its local copy.*
`git rm -r --cached folder/`	*Stop tracking a folder; keep local files.*

Amend rewrites the latest commit: use it on unpublished work or coordinate. After editing an already staged file, add it again to stage the new edits.

4. History, Comparison, And Search

COMMAND	MEANING
`git log`	*Show commit history.*
`git log --oneline -5`	*Show the latest five commits compactly.*

COMMAND	MEANING
`git log --oneline --graph --decorate --all`	*Show history across branches with labels.*
`git log -p -- FILE`	*Show commits and patches affecting a file.*
`git log --follow -- FILE`	*Follow a file's history across renames.*
`git log --author="NAME"`	*Filter commits by author.*
`git log --since="2026-01-01"`	*Filter by date.*
`git log main..BRANCH`	*Show commits reachable from BRANCH but not main.*
`git show HASH`	*Inspect a commit and its changes.*
`git show HASH:FILE`	*Show a file's content at a commit.*
`git diff HASH1 HASH2`	*Compare two commits.*
`git diff main...BRANCH`	*Compare BRANCH with its merge base with main.*
`git diff --stat`	*Show a summary of unstaged changes.*
`git blame FILE`	*Show the last modifying commit for each line.*
`git grep -n "TEXT"`	*Search tracked files, showing line numbers.*
`git shortlog -sn`	*Summarize commit counts by author in current history.*
`git describe --tags`	*Describe HEAD using nearby reachable tags.*
`git ls-files`	*List tracked files.*
`git check-ignore -v FILE`	*Explain which ignore rule matches a file.*
`git rev-parse HEAD`	*Print the current commit hash.*
`git rev-parse --show-toplevel`	*Print the repository's root path.*

5. Branches And Switching

COMMAND	MEANING
`git branch`	*List local branches; * marks the current one.*
`git branch -a`	*List local and remote-tracking branches.*
`git branch -r`	*List remote-tracking branches.*
`git branch -vv`	*Show local branch tips and tracking information.*
`git branch BRANCH`	*Create a branch without switching to it.*
`git switch BRANCH`	*Switch to an existing branch.*
`git switch -c BRANCH`	*Create and switch to a branch.*

COMMAND	MEANING
`git switch -c BRANCH HASH`	*Create a branch at a chosen commit and switch.*
`git switch -`	*Switch to the previously checked-out branch.*
`git switch --track origin/BRANCH`	*Create a local tracking branch and switch to it.*
`git switch --detach HASH`	*Inspect a commit in detached HEAD state.*
`git branch -m NEW_NAME`	*Rename the current branch locally.*
`git branch -d BRANCH`	*Delete a local branch after the merge safety check.*
`git branch -D BRANCH`	*Force-delete a local branch, even if unmerged.*
`git branch --set-upstream-to=origin/BRANCH`	*Set tracking for the current local branch.*
`git checkout BRANCH`	*Older command for switching branches.*
`git checkout -b BRANCH`	*Older create-and-switch command.*

Uncommitted changes may travel across a switch, or Git may block the switch. Deleting a local branch does not delete a remote branch.

6. Merge And Conflict Resolution

COMMAND	MEANING
`git merge BRANCH`	*Integrate BRANCH into the current branch.*
`git merge --ff-only BRANCH`	*Allow only a fast-forward; refuse divergence.*
`git merge --no-ff BRANCH`	*Create a merge commit even if fast-forward is possible.*
`git merge --squash BRANCH`	*Prepare combined changes without a merge commit; run git commit to record the staged result.*
`git status`	*Find conflicted files.*
`git add FILE`	*Mark an edited conflict resolution as staged.*
`git merge --continue`	*Finish a merge after resolving and staging conflicts.*
`git merge --abort`	*Cancel an in-progress merge.*
`git mergetool`	*Launch a configured conflict-resolution tool.*

Switch to the destination branch before merging. Review and remove conflict markers before staging the resolved file.

7. Rebase And Apply Selected Commits

COMMAND	MEANING
`git rebase main`	*Replay current-branch work on top of main.*
`git rebase -i HEAD~3`	*Interactively edit/squash the last three commits.*
`git rebase --continue`	*Continue after resolving and staging conflicts.*
`git rebase --skip`	*Omit the currently conflicting commit; intentional only.*
`git rebase --abort`	*Cancel the rebase and restore its starting state.*
`git cherry-pick HASH`	*Apply a commit's changes as a new current-branch commit.*
`git cherry-pick --no-commit HASH`	*Apply changes without automatically committing.*
`git cherry-pick --continue`	*Continue after resolving conflicts and staging files.*
`git cherry-pick --abort`	*Cancel the in-progress cherry-pick sequence.*

Rebase rewrites replayed commits. Coordinate before rewriting shared history. HEAD~3 assumes at least three parent steps exist; ordinary examples are linear.

8. Stash Unfinished Work

COMMAND	MEANING
`git stash`	*Save tracked staged/unstaged changes and set them aside.*
`git stash push -m "MESSAGE"`	*Save a named stash.*
`git stash push -u -m "MESSAGE"`	*Include untracked files, but not ignored files.*
`git stash push -a`	*Include untracked and ignored files too.*
`git stash list`	*List saved stashes.*
`git stash show -p`	*Show the latest stash's tracked patch.*
`git stash show -p -u`	*Include untracked files when displaying a stash.*
`git stash apply`	*Restore latest stash and keep its entry.*
`git stash apply --index`	*Also attempt to restore the staged state.*
`git stash apply 'stash@{1}'`	*Restore the second-most-recent stash.*
`git stash pop`	*Restore latest stash; delete entry only if successful.*
`git stash drop 'stash@{0}'`	*Delete a chosen stash entry.*
`git stash clear`	*Delete all stash entries; destructive.*

COMMAND	MEANING
`git stash branch rescue 'stash@{0}'`	*Create a branch at the stash's base and restore it; removes the entry on success.*

Stashes are local, may be applied on another branch, and may conflict.

9. Remotes, Fetch, Pull, And Push

COMMAND	MEANING
`git remote -v`	*Show remote URLs.*
`git remote add origin URL`	*Add a remote named origin.*
`git remote add upstream URL`	*Add the original repository when working from a fork.*
`git remote set-url origin URL`	*Change origin's URL.*
`git remote rename origin NEW_NAME`	*Rename a configured remote.*
`git remote remove NAME`	*Remove a remote configuration, not the hosted repo.*
`git remote show origin`	*Inspect a remote and its tracking information.*
`git fetch origin`	*Download history and update remote-tracking references.*
`git fetch --all`	*Fetch from all configured eligible remotes.*
`git fetch --prune origin`	*Also remove stale remote-tracking branch references.*
`git fetch --tags`	*Fetch remote tags.*
`git pull --ff-only`	*Fetch and fast-forward; refuse divergent histories.*
`git pull --no-rebase`	*Fetch and integrate using merge.*
`git pull --rebase`	*Fetch and integrate using rebase.*
`git push -u origin BRANCH`	*Push a branch and set its upstream tracking branch.*
`git push`	*Push according to configuration/tracking.*
`git push origin BRANCH`	*Push a particular branch.*
`git push origin --delete BRANCH`	*Delete a remote branch; affects collaborators.*
`git push origin v1.0`	*Publish a specific tag.*
`git push origin --tags`	*Publish all local tags.*
`git push --dry-run`	*Preview a push without updating remote refs.*
`git push --force-with-lease origin BRANCH`	*Replace remote branch history only if the lease check passes; coordinate before rewriting shared history.*

Fetch leaves your working files/current branch unchanged. Pull integrates fetched history and need not create a commit. Push sends commits, not unstaged or merely staged edits. Do not force-push just to fix a rejected normal push.

10. Undo And Recovery

COMMAND	MEANING
`git restore --staged FILE`	*Unstage a file while retaining working edits.*
`git restore FILE`	*Discard unstaged tracked edits; restore from the index.*
`git restore --source=HASH -- FILE`	*Restore that file's working content from a commit.*
`git restore --source=HEAD --staged --worktree FILE`	*Restore both staged and working content; discards edits.*
`git reset --soft HEAD~1`	*Undo the latest commit; keep its changes staged.*
`git reset --mixed HEAD~1`	*Undo it; keep changes in working files, but unstage them.*
`git reset --hard HASH`	*Reset HEAD, index, and tracked working files to HASH; discards local changes and may overwrite obstructing files.*
`git revert HASH`	*Create a new commit reversing an earlier commit.*
`git revert --continue`	*Continue a revert after resolving and staging conflicts.*
`git revert --abort`	*Cancel an in-progress revert sequence.*
`git reflog`	*Inspect local reference movements for recovery.*
`git branch rescue HASH`	*Preserve a recovered commit with a branch pointer.*
`git clean -nd`	*Preview deletion of untracked files/directories.*
`git clean -fd`	*Delete untracked files/directories, excluding ignored ones.*
`git clean -ndx`	*Preview deletion including ignored files/directories.*
`git clean -fdx`	*Delete untracked and ignored files/directories.*

Restore, hard reset, clean, and stash deletion can destroy local work. Preview clean first. Reset/amend rewrite branch history; revert preserves prior history. Reflog cannot recover every uncommitted edit or a file deleted by clean.

11. Tags

COMMAND	MEANING
`git tag`	*List tags.*
`git tag v1.0`	*Create a lightweight tag at HEAD.*
`git tag -a v1.0 -m "Release 1.0"`	*Create an annotated tag with metadata.*
`git tag -a v1.0 HASH -m "Release 1.0"`	*Tag a chosen commit.*
`git show v1.0`	*Inspect the tag and referenced commit.*
`git tag -d v1.0`	*Delete a local tag.*
`git push origin v1.0`	*Publish the tag.*
`git push origin --delete v1.0`	*Delete a remote tag; affects others using it.*
`git tag -s v1.0 -m "Release 1.0"`	*Create a signed tag; requires signing setup.*
`git tag -v v1.0`	*Verify a signed tag using available keys.*

12. Debugging With Bisect

COMMAND	MEANING
`git bisect start`	*Start a search for a bug-introducing commit.*
`git bisect bad`	*Mark the current commit as containing the bug.*
`git bisect good HASH`	*Mark a known earlier commit as working.*
`git bisect good`	*After testing, mark the checked-out commit as good.*
`git bisect bad`	*After testing, mark it as bad instead.*
`git bisect reset`	*Finish and return to the starting checkout.*

Repeat testing and good/bad until Git identifies the likely first bad commit.

13. Multiple Worktrees, Submodules, And Sparse Checkout

COMMAND	MEANING
`git worktree list`	*List working directories attached to the repo.*
`git worktree add -b hotfix ../hotfix main`	*Create a hotfix branch in a separate working directory.*
`git worktree remove ../hotfix`	*Remove that working directory (normally must be clean).*
`git worktree prune`	*Remove stale worktree metadata.*
`git submodule add URL folder`	*Add another repository as a tracked submodule.*

COMMAND	MEANING
`git submodule update --init --recursive`	*Initialize/check out recorded submodule commits.*
`git submodule status`	*Show recorded/checked-out submodule information.*
`git sparse-checkout init --cone`	*Enable directory-based sparse checkout.*
`git sparse-checkout set src docs`	*Limit checkout to those directories (plus root files).*
`git sparse-checkout disable`	*Restore the full working tree.*

14. Patches, Archives, And Maintenance

COMMAND	MEANING
`git diff > changes.patch`	*Save an unstaged tracked diff as a patch (shell redirect).*
`git apply --check changes.patch`	*Check whether a patch can be applied.*
`git apply changes.patch`	*Apply a patch without creating a commit.*
`git format-patch -1 HEAD`	*Export the latest commit as a mail-style patch.*
`git am PATCH_FILE`	*Apply a mail-style patch as a commit.*
`git am --continue`	*Continue after resolving an am conflict.*
`git am --abort`	*Cancel an in-progress am sequence.*
`git archive --format=zip --output=project.zip HEAD`	*Export tracked files at HEAD without Git history.*
`git bundle create backup.bundle --all`	*Save objects reachable from refs in an offline bundle.*
`git bundle verify backup.bundle`	*Check a bundle and its prerequisites.*
`git fsck`	*Check repository object connectivity and validity.*
`git count-objects -vH`	*Show object storage statistics.*
`git gc`	*Optimize repository object storage.*

Bundles/archives do not include arbitrary uncommitted working files.

15. Github: Publish Notes To A New, Empty Repository

Create an empty GitHub repository first, then inside your notes folder:

COMMAND	MEANING
`git init -b main`	
`git add README.md Git-Commands.txt Git-GitHub-Notes-Corrected.txt`	
`git commit -m "Add Git and GitHub study notes"`	

COMMAND	MEANING
`git remote add origin https://github.com/YOUR_USERNAME/git-notes.git`	
`git push -u origin main`	

For an existing repository: clone it, copy the files in, review git diff, stage them, commit, and push. Avoid unintentionally replacing its README.

Forking and opening/reviewing PRs are GitHub features, not core Git commands. You can use GitHub's website; the optional GitHub CLI uses gh, not git:

COMMAND	MEANING
`gh repo fork OWNER/REPO --clone`	*Fork and clone (requires GitHub CLI/authentication).*
`gh pr create`	*Open a pull request from your branch.*
`gh pr status`	*Show relevant pull requests.*

16. All-Command Catalogue

The following is generated by git help -a for the version shown below. It lists command names and descriptions, including advanced/internal helpers. gitk, scalar, and some graphical tools may be standalone executables or optional installations. Documentation topics at the end are help subjects, not executable subcommands. External commands depend on installed extensions. Use git help COMMAND for syntax; git COMMAND -h for short help.

git version 2.51.1

See 'git help <command>' to read about a specific subcommand

Main Porcelain Commands

COMMAND	MEANING
`add`	*Add file contents to the index*
`am`	*Apply a series of patches from a mailbox*
`archive`	*Create an archive of files from a named tree*
`backfill`	*Download missing objects in a partial clone*
`bisect`	*Use binary search to find the commit that introduced a bug*
`branch`	*List, create, or delete branches*
`bundle`	*Move objects and refs by archive*
`checkout`	*Switch branches or restore working tree files*
`cherry-pick`	*Apply the changes introduced by some existing commits*
`citool`	*Graphical alternative to git-commit*
`clean`	*Remove untracked files from the working tree*
`clone`	*Clone a repository into a new directory*
`commit`	*Record changes to the repository*
`describe`	*Give an object a human readable name based on an available ref*
`diff`	*Show changes between commits, commit and working tree, etc*
`fetch`	*Download objects and refs from another repository*
`format-patch`	*Prepare patches for e-mail submission*
`gc`	*Cleanup unnecessary files and optimize the local repository*

COMMAND	MEANING
`gitk`	*The Git repository browser*
`grep`	*Print lines matching a pattern*
`gui`	*A portable graphical interface to Git*
`init`	*Create an empty Git repository or reinitialize an existing one*
`log`	*Show commit logs*
`maintenance`	*Run tasks to optimize Git repository data*
`merge`	*Join two or more development histories together*
`mv`	*Move or rename a file, a directory, or a symlink*
`notes`	*Add or inspect object notes*
`pull`	*Fetch from and integrate with another repository or a local branch*
`push`	*Update remote refs along with associated objects*
`range-diff`	*Compare two commit ranges (e.g. two versions of a branch)*
`rebase`	*Reapply commits on top of another base tip*
`reset`	*Reset current HEAD to the specified state*
`restore`	*Restore working tree files*
`revert`	*Revert some existing commits*
`rm`	*Remove files from the working tree and from the index*
`scalar`	*A tool for managing large Git repositories*
`shortlog`	*Summarize 'git log' output*
`show`	*Show various types of objects*
`sparse-checkout`	*Reduce your working tree to a subset of tracked files*
`stash`	*Stash the changes in a dirty working directory away*
`status`	*Show the working tree status*
`submodule`	*Initialize, update or inspect submodules*
`switch`	*Switch branches*
`tag`	*Create, list, delete or verify a tag object signed with GPG*
`worktree`	*Manage multiple working trees*

Ancillary Commands / Manipulators

COMMAND	MEANING
`config`	*Get and set repository or global options*
`fast-export`	*Git data exporter*
`fast-import`	*Backend for fast Git data importers*
`filter-branch`	*Rewrite branches*
`mergetool`	*Run merge conflict resolution tools to resolve merge conflicts*
`pack-refs`	*Pack heads and tags for efficient repository access*
`prune`	*Prune all unreachable objects from the object database*
`reflog`	*Manage reflog information*
`refs`	*Low-level access to refs*
`remote`	*Manage set of tracked repositories*
`repack`	*Pack unpacked objects in a repository*
`replace`	*Create, list, delete refs to replace objects*

Ancillary Commands / Interrogators

COMMAND	MEANING
`annotate`	*Annotate file lines with commit information*
`blame`	*Show what revision and author last modified each line of a file*
`bugreport`	*Collect information for user to file a bug report*
`count-objects`	*Count unpacked number of objects and their disk consumption*
`diagnose`	*Generate a zip archive of diagnostic information*
`difftool`	*Show changes using common diff tools*
`fsck`	*Verifies the connectivity and validity of the objects in the database*
`gitweb`	*Git web interface (web frontend to Git repositories)*
`help`	*Display help information about Git*
`instaweb`	*Instantly browse your working repository in gitweb*

COMMAND	MEANING
merge-tree	*Perform merge without touching index or working tree*
rerere	*Reuse recorded resolution of conflicted merges*
show-branch	*Show branches and their commits*
verify-commit	*Check the GPG signature of commits*
verify-tag	*Check the GPG signature of tags*
version	*Display version information about Git*
whatchanged	*Show logs with differences each commit introduces*

Interacting with Others

COMMAND	MEANING
archimport	*Import a GNU Arch repository into Git*
cvsexportcommit	*Export a single commit to a CVS checkout*
cvsimport	*Salvage your data out of another SCM people love to hate*
cvsserver	*A CVS server emulator for Git*
imap-send	*Send a collection of patches from stdin to an IMAP folder*
p4	*Import from and submit to Perforce repositories*
quiltimport	*Applies a quilt patchset onto the current branch*
request-pull	*Generates a summary of pending changes*
send-email	*Send a collection of patches as emails*
svn	*Bidirectional operation between a Subversion repository and Git*

Low-level Commands / Manipulators

COMMAND	MEANING
apply	*Apply a patch to files and/or to the index*
checkout-index	*Copy files from the index to the working tree*
commit-graph	*Write and verify Git commit-graph files*
commit-tree	*Create a new commit object*

COMMAND	MEANING
`hash-object`	*Compute object ID and optionally create an object from a file*
`index-pack`	*Build pack index file for an existing packed archive*
`merge-file`	*Run a three-way file merge*
`merge-index`	*Run a merge for files needing merging*
`mktag`	*Creates a tag object with extra validation*
`mktree`	*Build a tree-object from ls-tree formatted text*
`multi-pack-index`	*Write and verify multi-pack-indexes*
`pack-objects`	*Create a packed archive of objects*
`prune-packed`	*Remove extra objects that are already in pack files*
`read-tree`	*Reads tree information into the index*
`replay`	*EXPERIMENTAL: Replay commits on a new base, works with bare repos too*
`symbolic-ref`	*Read, modify and delete symbolic refs*
`unpack-objects`	*Unpack objects from a packed archive*
`update-index`	*Register file contents in the working tree to the index*
`update-ref`	*Update the object name stored in a ref safely*
`write-tree`	*Create a tree object from the current index*

Low-level Commands / Interrogators

COMMAND	MEANING
`cat-file`	*Provide contents or details of repository objects*
`cherry`	*Find commits yet to be applied to upstream*
`diff-files`	*Compares files in the working tree and the index*
`diff-index`	*Compare a tree to the working tree or index*
`diff-pairs`	*Compare the content and mode of provided blob pairs*
`diff-tree`	*Compares the content and mode of blobs found via two tree objects*
`for-each-ref`	*Output information on each ref*
`for-each-repo`	*Run a Git command on a list of repositories*
`get-tar-commit-id`	*Extract commit ID from an archive created using git-archive*

COMMAND	MEANING
`ls-files`	*Show information about files in the index and the working tree*
`ls-remote`	*List references in a remote repository*
`ls-tree`	*List the contents of a tree object*
`merge-base`	*Find as good common ancestors as possible for a merge*
`name-rev`	*Find symbolic names for given revs*
`pack-redundant`	*Find redundant pack files*
`rev-list`	*Lists commit objects in reverse chronological order*
`rev-parse`	*Pick out and massage parameters*
`show-index`	*Show packed archive index*
`show-ref`	*List references in a local repository*
`unpack-file`	*Creates a temporary file with a blob's contents*
`var`	*Show a Git logical variable*
`verify-pack`	*Validate packed Git archive files*

Low-level Commands / Syncing Repositories

COMMAND	MEANING
`daemon`	*A really simple server for Git repositories*
`fetch-pack`	*Receive missing objects from another repository*
`http-backend`	*Server side implementation of Git over HTTP*
`send-pack`	*Push objects over Git protocol to another repository*
`update-server-info`	*Update auxiliary info file to help dumb servers*

Low-level Commands / Internal Helpers

COMMAND	MEANING
`check-attr`	*Display gitattributes information*
`check-ignore`	*Debug gitignore / exclude files*
`check-mailmap`	*Show canonical names and email addresses of contacts*
`check-ref-format`	*Ensures that a reference name is well formed*

COMMAND	MEANING
`column`	*Display data in columns*
`credential`	*Retrieve and store user credentials*
`credential-cache`	*Helper to temporarily store passwords in memory*
`credential-store`	*Helper to store credentials on disk*
`fmt-merge-msg`	*Produce a merge commit message*
`hook`	*Run git hooks*
`interpret-trailers`	*Add or parse structured information in commit messages*
`mailinfo`	*Extracts patch and authorship from a single e-mail message*
`mailsplit`	*Simple UNIX mbox splitter program*
`merge-one-file`	*The standard helper program to use with git-merge-index*
`patch-id`	*Compute unique ID for a patch*
`sh-i18n`	*Git's i18n setup code for shell scripts*
`sh-setup`	*Common Git shell script setup code*
`stripspace`	*Remove unnecessary whitespace*

User-facing repository, command and file interfaces

COMMAND	MEANING
`attributes`	*Defining attributes per path*
`cli`	*Git command-line interface and conventions*
`hooks`	*Hooks used by Git*
`ignore`	*Specifies intentionally untracked files to ignore*
`mailmap`	*Map author/committer names and/or E-Mail addresses*
`modules`	*Defining submodule properties*
`repository-layout`	*Git Repository Layout*
`revisions`	*Specifying revisions and ranges for Git*

Developer-facing file formats, protocols and other interfaces

COMMAND	MEANING
`format-bundle`	The bundle file format
`format-chunk`	Chunk-based file formats
`format-commit-graph`	Git commit-graph format
`format-index`	Git index format
`format-pack`	Git pack format
`format-signature`	Git cryptographic signature formats
`protocol-capabilities`	Protocol v0 and v1 capabilities
`protocol-common`	Things common to various protocols
`protocol-http`	Git HTTP-based protocols
`protocol-pack`	How packs are transferred over-the-wire
`protocol-v2`	Git Wire Protocol, Version 2

External commands

COMMAND	MEANING
`lfs`	Optional external extension.

Official references

Git command reference: https://git-scm.com/docs Complete Git manual: https://git-scm.com/docs/git GitHub documentation: https://docs.github.com GitHub CLI manual (optional): https://cli.github.com/manual/